

TRƯỜNG: ĐH CÔNG THƯƠNG TP.HCM KHOA: CÔNG NGHỆ THÔNG TIN BỘ MÔN: KHOA HỌC DỮ LIỆU MH: Lập trình Python	BUỔI 5. Tính đóng gói	
--	------------------------------	---

A. MỤC TIÊU

Sau khi học xong buổi học sinh viên có thể:

- Trình bày được khái niệm và vai trò của tính đóng gói (**Encapsulation**) trong lập trình hướng đối tượng.
- Phân biệt được các mức truy cập **public**, **protected** và **private** trong Python; giải thích được ý nghĩa của cách khai báo **_name** và **__name**.
- Khai báo và sử dụng được các thuộc tính **protected**, **private** để hạn chế việc truy cập và thay đổi dữ liệu trực tiếp từ bên ngoài lớp.
- Xây dựng và sử dụng được các phương thức **getter**, **setter** để truy xuất, cập nhật và kiểm tra tính hợp lệ của dữ liệu trước khi thay đổi thuộc tính.
- Sử dụng được **@property** và **setter** để xây dựng cơ chế truy cập thuộc tính theo cách phù hợp với Python.
- Vận dụng tính đóng gói để xây dựng các lớp có khả năng bảo vệ dữ liệu, kiểm soát thao tác cập nhật và xử lý các ràng buộc dữ liệu trong những bài toán thực tế.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1.1. Tính đóng gói và ẩn thông tin

Tính đóng gói (Encapsulation) là một trong bốn nguyên lý cơ bản của lập trình hướng đối tượng (OOP), giúp bảo vệ dữ liệu và ẩn đi các chi tiết triển khai bên trong của đối tượng.

Đóng gói là quá trình gói dữ liệu (thuộc tính) và các phương thức thao tác trên dữ liệu đó vào bên trong một lớp (class), che giấu chi tiết cài đặt và cung cấp giao diện để tương tác.

Ẩn thông tin là phần quan trọng trong đóng gói. Nó che giấu trạng thái nội tại của đối tượng khỏi bên ngoài, ngăn việc thay đổi dữ liệu trực tiếp mà không qua kiểm soát.

1.1.1. Biến **private** (**_name**) và **protected** (**_name**)

Trong Python, `private` và `protected` là hai cách để đánh dấu các biến (thuộc tính) hoặc phương thức trong lớp, giúp hạn chế truy cập từ bên ngoài lớp. Biến `private` và `protected` không được kiểm soát chặt như trong các ngôn ngữ như Java hay C++ mà được quy ước.

Protected Member (`_name`): Biến hoặc phương thức được đánh dấu là `protected` bằng cách thêm dấu gạch dưới `_` ở đầu tên (ví dụ: `_name`). Đây là một quy ước trong Python, ngụ ý rằng biến/phương thức này chỉ nên được truy cập bên trong lớp hoặc các lớp con (subclass), nhưng không nên truy cập từ bên ngoài.

Ví dụ 1.1.

```
class Person:
    def __init__(self, name):
        self._name = name # Protected member

    def get_name(self):
        return self._name # Truy cập protected bên trong lớp

p = Person("Alice")

print(p.get_name()) # Output: Alice
print(p._name)      # Vẫn truy cập được nhưng không nên
```

Private Member (`__name`): Biến `private` được định nghĩa với hai dấu gạch dưới `__` ở đầu tên biến (ví dụ `__name`). Biến này không thể truy cập trực tiếp từ bên ngoài lớp do Python áp dụng kỹ thuật gọi là `name mangling`.

Muốn truy cập đến biến `private` phải thông qua phương thức công khai (getter/setter) hoặc sử dụng cú pháp `_TênLớp_tên`.

Ví dụ 1.2.

```
class Person:
    def __init__(self, name):
        self.__name = name # Private member

    def get_name(self): # Getter để truy cập private member
        return self.__name

p = Person("Alice")
# print(p.__name) # Lỗi: AttributeError
print(p.get_name()) # Output: Alice (truy cập qua getter)
print(p._Person__name) # Truy cập được nhưng không khuyến khích
```

1.1.2. Getter, Setter

Getter và setter là các phương thức được sử dụng để truy cập (lấy giá trị) và thay đổi (gán giá trị) các thuộc tính (attributes) của một lớp, đặc biệt là các thuộc tính `private` hoặc `protected`. Điều này giúp thực hiện đóng gói (encapsulation), đảm bảo rằng dữ liệu bên trong lớp được kiểm soát và bảo vệ khỏi truy cập không hợp lệ.

getter: là phương thức dùng để lấy giá trị của một thuộc tính. Nó cho phép truy cập gián tiếp vào thuộc tính mà không cần truy cập trực tiếp.

Ví dụ 1.3.

```
class Person:
    def __init__(self, name):
        self.__name = name # Private attribute
    # Getter cho __name
    def get_name(self):
        return self.__name
p = Person("Alice")
print(p.get_name()) # Output: Alice
```

setter: là phương thức dùng để gán giá trị mới cho một thuộc tính. Nó cho phép kiểm tra hoặc xử lý giá trị trước khi gán.

Ví dụ 1.4.

```
class Person:
    def __init__(self, name):
        self.__name = name

    # Setter cho __name (kiểm tra tên không rỗng)
    def set_name(self, new_name):
        if new_name.strip(): # Loại bỏ khoảng trắng
            self.__name = new_name
        else:
            print("Tên không hợp lệ!")

p = Person("Alice")
p.set_name("Bob")
p.set_name("") # Output: Tên không hợp lệ!
```

Getter và setter thường được sử dụng để:

- Ẩn chi tiết triển khai của lớp (encapsulation).
- Kiểm tra dữ liệu đầu vào (validation) trước khi thay đổi thuộc tính.
- Thêm logic bổ sung khi lấy hoặc gán giá trị.

1.1.3. Khái niệm `@property` để tạo getter/setter Pythonic

Python cung cấp cách viết ngắn gọn và hiện đại hơn thông qua decorator `@property` để định nghĩa getter và setter.

Trong đó:

- **`@property`**: Định nghĩa getter.

- **@<tên>.setter**: Định nghĩa setter.

Ví dụ 1.5.

```
class Person:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self): # Getter
        return self.__name

    @name.setter
    def name(self, new_name): # Setter
        if new_name.strip():
            self.__name = new_name
        else:
            print("Tên không hợp lệ!")

p = Person("Alice")
print(p.name)          # Output: Alice (gọi getter)
p.name = "Bob"         # Gọi setter
print(p.name)          # Output: Bob
p.name = ""            # Output: Tên không hợp lệ!
```

A. BÀI TẬP TRÊN LỚP:

Bài 1. Lớp Person

Xây dựng lớp Person dùng để lưu trữ và xử lý thông tin của một người. Các thuộc tính của lớp được đóng gói, không cho phép chương trình bên ngoài thay đổi tùy ý.

Thuộc tính:

- `__ho_ten`: họ và tên, là thuộc tính private.
- `_quoc_gia`: quốc gia, là thuộc tính protected.
- `__ngay_sinh`: ngày sinh, là thuộc tính private.

Các phương thức:

1. `__init__(self, ho_ten, quoc_gia, ngay_sinh)`
 - Khởi tạo họ tên, quốc gia và ngày sinh.
2. `get_ho_ten(self)`
 - Trả về họ tên của người.
3. `set_ho_ten(self, ho_ten)`
 - Cập nhật họ tên.
 - Không chấp nhận họ tên rỗng.
4. `get_ngay_sinh(self)`
 - Trả về ngày sinh.
5. `set_ngay_sinh(self, ngay_sinh)`

- Cập nhật ngày sinh.
 - Ngày sinh không được lớn hơn ngày hiện tại.
6. `calculate_age(self)`
- Tính tuổi hiện tại dựa trên ngày sinh và ngày hiện tại.
 - Trả về số tuổi đã đủ.
7. `display(self)`
- Hiển thị:
 - Họ tên.
 - Quốc gia.
 - Ngày sinh.
 - Tuổi hiện tại.

Yêu cầu chương trình chính:

- (1) Tạo ít nhất một đối tượng thuộc lớp `Person`.
- (2) Gọi `display()` để hiển thị thông tin.
- (3) Sử dụng `get_ho_ten()` để đọc họ tên.
- (4) Sử dụng `set_ho_ten()` để thay đổi họ tên.
- (5) Thử truy cập trực tiếp thuộc tính `_quoc_gia` và nhận xét.
- (6) Thử truy cập trực tiếp thuộc tính `__ho_ten` và nhận xét kết quả.

Bài 2. Lớp Khách hàng

Xây dựng lớp `Customer` dùng để lưu thông tin khách hàng và tính số tiền cần thanh toán. Các thông tin của khách hàng được đóng gói để hạn chế việc thay đổi trực tiếp từ bên ngoài lớp.

Thuộc tính:

- `__ma_kh`: mã khách hàng.
- `__ho_ten`: họ tên khách hàng.
- `__loai_kh`: loại khách hàng.
- `__tong_tien`: tổng tiền mua hàng.

Quy định loại khách hàng và mức giảm giá:

- Thường: 0%.
- Bạc: 5%.
- Vàng: 10%.
- VIP: 15%.

Các phương thức:

1. `__init__(self, ma_kh, ho_ten, loai_kh, tong_tien)`
 - Khởi tạo thông tin khách hàng.
 - Tổng tiền mua hàng phải lớn hơn hoặc bằng 0.
 - Loại khách hàng phải thuộc một trong các loại: Thường, Bạc, Vàng, VIP.
2. `get_ho_ten(self)`
 - Trả về họ tên khách hàng.

3. `set_ho_ten(self, ho_ten)`
 - Cập nhật họ tên khách hàng.
 - Không chấp nhận họ tên rỗng.
4. `get_tong_tien(self)`
 - Trả về tổng tiền mua hàng.
5. `set_tong_tien(self, tong_tien)`
 - Cập nhật tổng tiền mua hàng.
 - Chỉ chấp nhận giá trị lớn hơn hoặc bằng 0.
6. `discount_rate(self)`
 - Xác định và trả về tỷ lệ giảm giá tương ứng với loại khách hàng.
7. `discount_amount(self)`
 - Tính số tiền được giảm theo công thức:

$$\text{Tiền giảm} = \text{Tổng tiền mua hàng} \times \text{Tỷ lệ giảm}$$
8. `payment(self)`
 - Tính số tiền thực tế khách hàng phải thanh toán:

$$\text{Thanh toán} = \text{Tổng tiền mua hàng} - \text{Tiền giảm}$$
9. `display(self)`
 - Hiển thị:
 - Mã khách hàng.
 - Họ tên.
 - Loại khách hàng.
 - Tổng tiền mua hàng.
 - Tỷ lệ giảm giá.
 - Số tiền được giảm.
 - Số tiền thực tế phải thanh toán.

Yêu cầu chương trình chính:

- (1) Tạo ít nhất một đối tượng thuộc lớp `Customer`.
- (2) Gọi `display()` để hiển thị thông tin.
- (3) Sử dụng `get_tong_tien()` để xem tổng tiền mua hàng.
- (4) Sử dụng `set_tong_tien()` để thay đổi tổng tiền mua hàng.
- (5) Thử truy cập trực tiếp thuộc tính `__tong_tien` từ bên ngoài lớp và nhận xét kết quả.

Bài 3. Lớp Số phức

Xây dựng lớp `ComplexNumber` dùng để biểu diễn và thực hiện một số phép toán cơ bản với số phức.

Một số phức có dạng:

$$a + bi$$

Trong đó:

- a là phần thực.
- b là phần ảo.

Thuộc tính:

- `__real__`: lưu phần thực của số phức.
- `__imag__`: lưu phần ảo của số phức.

Hai thuộc tính trên được khai báo là **private** để thể hiện tính đóng gói.

Các phương thức:

1. `__init__(self, real=0, imag=0)`
 - Khởi tạo số phức với phần thực và phần ảo.
 - Nếu không truyền giá trị, phần thực và phần ảo mặc định bằng 0.
2. `get_real(self)`
 - Trả về phần thực của số phức.
3. `set_real(self, real)`
 - Cập nhật phần thực của số phức.
4. `get_imag(self)`
 - Trả về phần ảo của số phức.
5. `set_imag(self, imag)`
 - Cập nhật phần ảo của số phức.
6. `input_number(self)`
 - Nhập phần thực và phần ảo từ bàn phím.
 - Cập nhật các giá trị vừa nhập cho đối tượng hiện tại thông qua các phương thức setter.
7. `display(self)`
 - Hiển thị số phức theo dạng: $a + bi$.
 - Nếu phần ảo là số âm thì hiển thị theo dạng: $a - bi$.
8. `add(self, other)`
 - Cộng số phức hiện tại với số phức other.
 - Công thức:

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$

- Trả về một đối tượng ComplexNumber mới chứa kết quả phép cộng.

9. `subtract(self, other)`
 - Lấy số phức hiện tại trừ số phức other.
 - Công thức:

$$(a + bi) - (c + di) = (a - c) + (b - d)i$$

- Trả về một đối tượng ComplexNumber mới chứa kết quả phép trừ.

Yêu cầu chương trình chính:

- (1) Tạo hai đối tượng số phức.

- (2) Nhập phần thực và phần ảo cho từng đối tượng bằng phương thức `input_number()`.
- (3) Hiển thị hai số phức đã nhập.
- (4) Sử dụng getter để lấy phần thực hoặc phần ảo của một số phức.
- (5) Sử dụng setter để thay đổi phần thực hoặc phần ảo.
- (6) Thực hiện phép cộng hai số phức và hiển thị kết quả.
- (7) Thực hiện phép trừ hai số phức và hiển thị kết quả.
- (8) Thử truy cập trực tiếp thuộc tính `__real` hoặc `__imag` từ bên ngoài lớp và nhận xét kết quả.

Bài 4. Lớp Danh sách số nguyên

Xây dựng lớp `IntegerList` dùng để quản lý một danh sách các số nguyên.

Thuộc tính:

- `numbers`: danh sách số nguyên.

Các phương thức:

- `__init__()`: khởi tạo danh sách rỗng.
- `add_number(number)`: thêm một số vào danh sách.
- `remove_number(number)`: xóa một số khỏi danh sách.
- `search_number(number)`: kiểm tra số có tồn tại trong danh sách hay không.
- `sum_numbers()`: tính tổng các phần tử.
- `maximum()`: tìm phần tử lớn nhất.
- `minimum()`: tìm phần tử nhỏ nhất.
- `display()`: hiển thị toàn bộ danh sách.

Yêu cầu: Xử lý trường hợp danh sách rỗng hoặc số cần xóa không tồn tại.

Bài 5. Quản lý nhân viên

Xây dựng hai lớp `Employee` và `EmployeeManager`.

Lớp `Employee`

Thuộc tính có thể gồm:

- Mã nhân viên.
- Họ tên.
- Phòng ban.
- Lương cơ bản.
- Số giờ làm.

Các phương thức:

- `__init__()`: khởi tạo thông tin nhân viên.
- `bonus()`: tính tiền thưởng dựa vào số giờ làm.
- `income()`: tính tổng thu nhập gồm lương cơ bản và thưởng.
- `display()`: hiển thị thông tin và thu nhập.

Lớp `EmployeeManager`

Các phương thức:

- `add_employee()`: thêm nhân viên mới, không cho trùng mã.
- `find_employee()`: tìm nhân viên theo mã.
- `update_salary()`: cập nhật lương cơ bản.
- `delete_employee()`: xóa nhân viên.
- `sort_by_income()`: sắp xếp nhân viên giảm dần theo thu nhập.
- `department_statistics()`: thống kê số nhân viên theo phòng ban.

Bài 6. Lớp Tài khoản ngân hàng

Xây dựng lớp `BankAccount` dùng để lưu trữ thông tin và thực hiện các giao dịch cơ bản của một tài khoản ngân hàng.

Thuộc tính:

- `__account_number`: số tài khoản.
- `__owner`: chủ tài khoản.
- `__balance`: số dư tài khoản.
- `__status`: trạng thái tài khoản, gồm Hoạt động hoặc Khóa.
- `__history`: danh sách lưu lịch sử giao dịch.

Các thuộc tính trên được khai báo là **private** để thể hiện tính đóng gói.

Các phương thức:

1. `__init__(self, account_number, owner, balance=0)`
 - Khởi tạo số tài khoản, chủ tài khoản và số dư.
 - Số dư ban đầu phải lớn hơn hoặc bằng 0.
 - Trạng thái mặc định là Hoạt động.
 - Lịch sử giao dịch ban đầu là danh sách rỗng.
2. `get_balance(self)`
 - Trả về số dư hiện tại của tài khoản.
3. `get_status(self)`
 - Trả về trạng thái hiện tại của tài khoản.
4. `deposit(self, amount)`
 - Gửi tiền vào tài khoản.
 - Số tiền gửi phải lớn hơn 0.
 - Tài khoản phải đang hoạt động.
 - Cập nhật số dư và lưu thông tin vào lịch sử giao dịch.
5. `withdraw(self, amount)`
 - Rút tiền khỏi tài khoản.
 - Số tiền rút phải lớn hơn 0.
 - Tài khoản phải đang hoạt động.
 - Số dư phải đủ để thực hiện giao dịch.
 - Cập nhật số dư và lịch sử giao dịch.
6. `transfer(self, other, amount)`
 - Chuyển tiền từ tài khoản hiện tại sang tài khoản `other`.

- Số tiền chuyển phải lớn hơn 0.
 - Cả hai tài khoản phải đang hoạt động.
 - Tài khoản chuyển phải có đủ số dư.
 - Cập nhật số dư và lịch sử giao dịch của hai tài khoản.
7. `lock_account(self)`
- Chuyển trạng thái tài khoản thành Khóa.
 - Khi tài khoản bị khóa thì không được gửi, rút hoặc chuyển tiền.
8. `unlock_account(self)`
- Chuyển trạng thái tài khoản về Hoạt động.
9. `show_history(self)`
- Hiện thị toàn bộ lịch sử giao dịch của tài khoản.
10. `display(self)`
- Hiện thị:
 - Số tài khoản.
 - Chủ tài khoản.
 - Số dư.
 - Trạng thái.

Yêu cầu chương trình chính:

- (1) Tạo hai đối tượng thuộc lớp `BankAccount`.
- (2) Gửi tiền vào một tài khoản.
- (3) Rút tiền từ một tài khoản.
- (4) Chuyển tiền giữa hai tài khoản.
- (5) Hiện thị số dư sau các giao dịch.
- (6) Khóa một tài khoản và thử thực hiện giao dịch để kiểm tra.
- (7) Hiện thị lịch sử giao dịch.
- (8) Thử truy cập trực tiếp `__balance` từ bên ngoài lớp và nhận xét kết quả.

Bài 7. Quản lý bãi giữ xe

Xây dựng lớp `Vehicle` để lưu thông tin xe và lớp `ParkingManager` để quản lý các xe ra vào bãi giữ xe.

Lớp Vehicle

Thông tin xe gồm:

- Biển số xe.
- Loại xe: xe đạp, xe máy hoặc ô tô.
- Thời gian vào bãi.
- Thời gian ra bãi.
- Trạng thái: đang gửi hoặc đã lấy.

Các phương thức:

- `__init__()`: khởi tạo thông tin xe.
- `calculate_fee()`: tính tiền gửi xe dựa vào loại xe và thời gian gửi.

- `display()`: hiển thị thông tin xe.

Có thể quy định đơn giá tham khảo:

- Xe đạp: 3.000 đồng/lượt.
- Xe máy: 5.000 đồng/lượt.
- Ô tô: 20.000 đồng/lượt.

Lớp ParkingManager

Các phương thức:

- `check_in()`: tiếp nhận xe vào bãi, không cho phép một biển số đang gửi được gửi thêm lần nữa.
- `find_vehicle()`: tìm xe theo biển số.
- `check_out()`: thực hiện cho xe ra khỏi bãi, cập nhật thời gian ra và tính tiền gửi xe.
- `display_vehicles()`: hiển thị danh sách các xe hiện có trong bãi.
- `find_by_type()`: tìm các xe theo loại xe.
- `count_vehicles()`: đếm số xe đang có trong bãi theo từng loại.
- `total_revenue()`: tính tổng tiền thu được từ các xe đã ra khỏi bãi.

Yêu cầu chương trình chính

- (1) Thêm một số xe vào bãi.
- (2) Không cho phép hai xe đang gửi có cùng biển số.
- (3) Tìm kiếm xe theo biển số.
- (4) Hiển thị các xe đang có trong bãi.
- (5) Thực hiện cho một xe ra khỏi bãi và tính tiền gửi xe.
- (6) Tìm các xe theo loại xe.
- (7) Thống kê số lượng xe đang gửi.
- (8) Tính tổng doanh thu từ các xe đã hoàn thành việc gửi.

Bài 8. Lớp Sách thư viện (LibraryBook)

Xây dựng lớp LibraryBook để quản lý thông tin sách trong thư viện trường học. Để bảo vệ dữ liệu sách khỏi việc sửa đổi tùy tiện từ bên ngoài, toàn bộ thuộc tính dữ liệu đều được đóng gói thành các thuộc tính private và kiểm soát cập nhật nghiêm ngặt.

Thuộc tính private:

`self.__ma_sach`: Mã sách (chuỗi). Ràng buộc: phải có độ dài đúng 5 ký tự và bắt đầu bằng ký tự 'S' (ví dụ: 'S0102').

`self.__ten_sach`: Tên sách (chuỗi). Ràng buộc: không được để rỗng.

`self.__tac_gia`: Họ tên tác giả (chuỗi). Ràng buộc: không được để rỗng.

`self.__don_gia`: Đơn giá sách (số thực). Ràng buộc: phải lớn hơn 0.

`self.__so_luong_ton`: Số lượng sách hiện có trong kho (số nguyên). Ràng buộc: phải lớn hơn hoặc bằng 0.

Các phương thức:

`__init__(self, ma_sach, ten_sach, tac_gia, don_gia, so_luong_ton=0)`: Khởi tạo thông tin sách. Áp dụng cơ chế kiểm tra tính hợp lệ của tất cả các tham số truyền vào trước khi gán (nếu dữ liệu không hợp lệ thì hiển thị thông báo lỗi và gán các giá trị mặc định hợp lệ).

Sử dụng decorator `@property` và setter cho `don_gia` và `so_luong_ton`: Để cho phép truy cập và cập nhật hai thuộc tính này một cách an toàn và Pythonic. Đảm bảo ném ra thông báo lỗi khi gán giá trị không hợp lệ (ví dụ gán số lượng âm hoặc đơn giá ≤ 0) và không thay đổi giá trị hiện tại.

Getter và setter cho `ten_sach` và `tac_gia`: Xây dựng các phương thức `get_ten_sach()`, `set_ten_sach()`, `get_tac_gia()`, `set_tac_gia()` truyền thống để thực hành linh hoạt cả hai cách tiếp cận.

`borrow_book(self, quantity)`: Thực hiện thao tác cho mượn sách. Đầu vào là số lượng sách cần mượn (số nguyên dương). Kiểm tra nếu số lượng tồn kho hiện tại đủ cung cấp thì giảm số lượng tồn đi tương ứng và thông báo mượn sách thành công. Ngược lại, thông báo lỗi không đủ sách tồn kho.

`restock(self, quantity)`: Thực hiện nhập thêm sách vào kho. Cộng thêm số lượng nhập (phải > 0) vào số lượng tồn kho.

`total_value(self)`: Tính toán và trả về tổng giá trị tồn kho của cuốn sách hiện tại (Tổng giá = Đơn giá $\times$ Số lượng tồn).

`display(self)`: Hiển thị đầy đủ thông tin sách bao gồm: Mã sách, Tên sách, Tác giả, Đơn giá, Số lượng tồn, và Tổng giá trị tồn kho.

Yêu cầu chương trình chính (Main program):

Tạo ra ít nhất 2 đối tượng sách thuộc lớp `LibraryBook` (một đối tượng khởi tạo hợp lệ, một đối tượng cố ý khởi tạo sai mã sách hoặc đơn giá để kiểm tra logic xử lý lỗi).

Gọi phương thức `display()` để kiểm tra thông tin hiển thị.

Sử dụng setter `@property` của đơn giá để cập nhật giá mới phù hợp.

Thử thay đổi thuộc tính số lượng tồn kho thành một số âm thông qua property setter để chứng minh tính đóng gói đã ngăn chặn được dữ liệu sai lệch.

Thực hiện mượn sách 2 lần (một lần mượn số lượng hợp lệ, một lần cố tình mượn quá số lượng tồn kho) và quan sát kết quả.

Nhập thêm sách bằng phương thức `restock()` rồi tính lại tổng giá trị tồn kho và hiển thị.

Bài 9. Lớp Phân số (Fraction)

Xây dựng lớp `Fraction` biểu diễn một phân số toán học. Trong toán học, mẫu số bắt buộc phải khác 0. Bài tập này giúp sinh viên hiểu rõ tầm quan trọng của việc ẩn trạng thái nội tại và ràng buộc dữ liệu nghiêm ngặt trong các mô hình thực thể toán học, kết hợp đóng gói cả phương thức hỗ trợ nội bộ (private method).

Thuộc tính private:

`self.__tu_so`: Tử số của phân số (số nguyên).

`self.__mau_so`: Mẫu số của phân số (số nguyên, bắt buộc phải khác 0, mặc định bằng 1).

Các phương thức:

`__init__(self, tu_so=0, mau_so=1)`: Khởi tạo phân số. Nếu người dùng truyền mẫu số bằng 0, in ra thông báo lỗi "Mẫu số không thể bằng 0. Hệ thống tự động đặt mẫu số là 1" và tiến hành gán `__mau_so = 1`.

Getter và setter cho cả tử số và mẫu số: Sử dụng cách viết getter/setter truyền thông. Đặc biệt trong phương thức `set_mau_so(self, value)`, nếu `value = 0` thì không cập nhật mẫu số, giữ nguyên giá trị cũ và thông báo lỗi ra màn hình.

Phương thức private `__gcd(self, a, b)`: Phương thức hỗ trợ nội bộ (được ẩn đi bằng hai dấu gạch dưới) dùng để tìm và trả về Ước chung lớn nhất (UCLN) của hai số nguyên `a` và `b` (sử dụng thuật toán Euclid). Phương thức này không cho phép gọi từ ngoài lớp.

`simplify(self)`: Rút gọn phân số hiện tại bằng cách chia cả tử và mẫu cho UCLN nhận được từ phương thức `__gcd()`. Đảm bảo chuẩn hóa dấu của phân số (nếu mẫu số âm, đưa dấu âm lên tử số và đổi mẫu số thành dương).

`add(self, other)`: Cộng phân số hiện tại với phân số `other`. Trả về một đối tượng Fraction mới đại diện cho phân số tổng sau khi đã rút gọn (tối giản). Công thức: $(a/b) + (c/d) = (a*d + b*c) / (b*d)$.

`multiply(self, other)`: Nhân phân số hiện tại với phân số `other`. Trả về một đối tượng Fraction mới đại diện cho phân số tích sau khi đã rút gọn. Công thức: $(a/b) * (c/d) = (a*c) / (b*d)$.

`display(self)`: Hiển thị phân số dưới dạng chuẩn hóa: `'tu_so/mau_so'`. Ràng buộc hiển thị đặc biệt: 1) Nếu tử số bằng 0, chỉ hiển thị số '0'. 2) Nếu mẫu số bằng 1, chỉ hiển thị giá trị 'tu_so' dưới dạng số nguyên.

Yêu cầu chương trình chính (Main program):

Khởi tạo 2 phân số `f1` và `f2` từ dữ liệu nhập vào (ví dụ `f1 = 3/6`, `f2 = -2/-4`).

Hiển thị `f1` và `f2` trước và sau khi gọi phương thức `simplify()` để kiểm tra thuật toán tối giản phân số.

Thực hiện phép tính tổng `f3 = f1 + f2` và tích `f4 = f1 * f2`. Hiển thị kết quả tối giản của `f3` và `f4`.

Thực hiện đổi mẫu số của `f1` thành 0 bằng setter `set_mau_so()` để chứng minh cơ chế đóng gói bảo vệ dữ liệu hoạt động chính xác.

Cố gắng gọi trực tiếp phương thức `f1.__gcd()` từ bên ngoài chương trình để kiểm chứng lỗi `AttributeError` do Python ngăn chặn việc truy cập phương thức private.

Bài 10. Lớp Sản phẩm điện tử (ElectronicProduct)

Xây dựng lớp `ElectronicProduct` quản lý sản phẩm trong hệ thống siêu thị điện máy. Bài thực hành này giúp sinh viên phân biệt sâu sắc giữa các mức độ truy cập `Public`, `Protected` và `Private`, đồng thời học cách bảo vệ các biến nhạy cảm như giá cả và bảo hành.

Thuộc tính:

`self.__ma_sp`: Mã sản phẩm (chuỗi, private).

`self.__ten_sp`: Tên sản phẩm (chuỗi, private).

`self._hang_sx`: Hãng sản xuất (chuỗi, protected).

self.__gia_nhap: Giá nhập sản phẩm (số thực, private). Ràng buộc: phải > 0.

self.__phan_tram_lai: Phần trăm lợi nhuận định mức để tính giá bán (số thực, private).

Ràng buộc: từ 0.05 (5%) đến 0.5 (50%), mặc định 0.1 (10%).

self.__thang_bao_hanh: Thời gian bảo hành tính bằng tháng (số nguyên, private). Ràng buộc: phải lớn hơn hoặc bằng 0, mặc định là 12.

Các phương thức:

__init__(self, ma_sp, ten_sp, hang_sx, gia_nhap, phan_tram_lai=0.1, thang_bao_hanh=12): Khởi tạo thông tin sản phẩm và gán giá trị sau khi đã đi qua kiểm tra tính hợp lệ.

Getter và setter bằng decorator @property cho gia_nhap, phan_tram_lai, thang_bao_hanh: Các setter phải chứa điều kiện kiểm tra nghiêm ngặt (ví dụ: gia_nhap > 0; phan_tram_lai nằm trong [0.05, 0.5]; thang_bao_hanh >= 0). Nếu không hợp lệ, giữ nguyên giá trị cũ và in thông báo cảnh báo lỗi.

calculate_selling_price(self): Tính giá bán lẻ của sản phẩm. Công thức: Giá bán = Giá nhập $\times$ (1 + Phần trăm lợi nhuận). Trả về giá trị số thực.

check_warranty(self, months_used): Nhận vào số tháng sản phẩm đã được khách hàng sử dụng (số nguyên >= 0). Trả về True nếu số tháng này nhỏ hơn hoặc bằng số tháng bảo hành của sản phẩm (còn hạn bảo hành), ngược lại trả về False.

display(self): Hiển thị thông tin sản phẩm chi tiết gồm: Mã SP, Tên SP, Hãng sản xuất, Giá nhập, Tỷ lệ lợi nhuận, Giá bán đề xuất, và Số tháng bảo hành.

Yêu cầu chương trình chính (Main program):

Tạo một đối tượng thuộc lớp ElectronicProduct (ví dụ: Tivi Samsung, giá nhập 15,000,000đ, bảo hành 24 tháng).

Hiển thị thông tin sản phẩm ban đầu bằng display() để xem giá bán lẻ đề xuất.

Thực hiện thay đổi giá nhập thông qua property setter và hiển thị lại xem giá bán lẻ thay đổi như thế nào.

Nhập số tháng đã sử dụng của khách hàng từ bàn phím, dùng check_warranty() để thông báo sản phẩm của khách hàng còn được nhận bảo hành miễn phí hay không.

Thực hiện truy cập trực tiếp biến protected __hang_sx từ bên ngoài lớp (ví dụ: print(sp._hang_sx)) và rút ra nhận xét về quy ước trong Python.

Cố gắng truy cập trực tiếp biến private __gia_nhap từ bên ngoài lớp (ví dụ: print(sp.__gia_nhap)) để xem lỗi, sau đó giải thích cách thức Python bảo vệ biến private qua cơ chế Name Mangling (truy cập thử bằng cú pháp lách luật _TênLớp__tenThuocTinh để đổi chiều).

B. BÀI TẬP VỀ NHÀ

Bài 1. Lớp Điểm trong mặt phẳng

Xây dựng lớp Point dùng để biểu diễn một điểm trong mặt phẳng tọa độ Oxy.

Thuộc tính:

- __x: hoành độ của điểm.

- `__y`: tung độ của điểm.

Hai thuộc tính được khai báo là private để thể hiện tính đóng gói.

Các phương thức:

1. `__init__(self, x=0, y=0)`
 - Khởi tạo tọa độ x, y cho điểm.
 - Nếu không truyền giá trị thì mặc định điểm có tọa độ (0, 0).
2. `get_x(self)`
 - Trả về hoành độ của điểm.
3. `set_x(self, x)`
 - Cập nhật hoành độ của điểm.
4. `get_y(self)`
 - Trả về tung độ của điểm.
5. `set_y(self, y)`
 - Cập nhật tung độ của điểm.
6. `distance_to_origin(self)`
 - Tính khoảng cách từ điểm hiện tại đến gốc tọa độ O(0, 0).
 - Công thức:

$$d = \sqrt{x^2 + y^2}$$

7. `distance_to(self, other)`
 - Tính khoảng cách từ điểm hiện tại đến điểm other.
 - Với hai điểm $A(x_1, y_1)$ và $B(x_2, y_2)$:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

8. `display(self)`
 - Hiển thị tọa độ điểm theo dạng:

(x, y)

Yêu cầu chương trình chính:

- (1) Tạo hai đối tượng thuộc lớp Point.
- (2) Hiển thị tọa độ của hai điểm.
- (3) Sử dụng getter để lấy hoành độ và tung độ.
- (4) Sử dụng setter để thay đổi tọa độ của một điểm.
- (5) Tính khoảng cách từ mỗi điểm đến gốc tọa độ.
- (6) Tính khoảng cách giữa hai điểm.
- (7) Thử truy cập trực tiếp thuộc tính `__x` hoặc `__y` từ bên ngoài lớp và nhận xét kết quả.

Bài 2. Lớp Nhiệt độ

Xây dựng lớp Temperature dùng để lưu nhiệt độ theo thang Celsius và thực hiện chuyển đổi sang các thang đo khác.

Thuộc tính:

- celsius: nhiệt độ theo độ C.

Các phương thức:

- `__init__(self, celsius)`: khởi tạo nhiệt độ Celsius, không cho phép nhỏ hơn -273.15°C .
- `to_fahrenheit()`: chuyển nhiệt độ từ Celsius sang Fahrenheit.
- `to_kelvin()`: chuyển nhiệt độ từ Celsius sang Kelvin.
- `display()`: hiển thị nhiệt độ theo cả ba thang Celsius, Fahrenheit và Kelvin.

Yêu cầu: Kiểm tra dữ liệu nhiệt độ hợp lệ trước khi thực hiện chuyển đổi.

Bài 3. Lớp Đồng hồ

Xây dựng lớp Clock dùng để biểu diễn thời gian.

Thuộc tính:

- hour: giờ.
- minute: phút.
- second: giây.

Các phương thức:

- `__init__()`: khởi tạo giờ, phút và giây.
- `is_valid()`: kiểm tra thời gian hợp lệ với điều kiện:
 - $0 \leq \text{hour} \leq 23$
 - $0 \leq \text{minute} \leq 59$
 - $0 \leq \text{second} \leq 59$
- `display()`: hiển thị thời gian theo định dạng hh:mm:ss.

Yêu cầu: Nếu thời gian không hợp lệ phải có thông báo phù hợp.

Bài 4: Lớp Student

Xây dựng lớp Student gồm các thông tin: Mã sinh viên, ngành học, Khóa, năm nhập học, điểm trung bình, điểm giữa kỳ, điểm cuối kỳ, điểm kiểm tra. Trong đó mã sinh viên là một chuỗi có 5 ký tự: 2 ký tự đầu là mã ngành học, 3 ký tự sau là số

- Xây dựng phương thức Nhập thông tin của Student.
- Xây dựng phương thức tính điểm trung bình của sinh viên (điểm trung bình = điểm giữa kỳ + điểm cuối kỳ + điểm kiểm tra)/3
- Xây dựng phương thức Xếp loại cho sinh viên, nhập vào điểm trung bình,
 - + Nếu điểm trung bình ≥ 8 là loại giỏi
 - + Điểm trung bình ≥ 6.5 & < 8 là loại khá,
 - + Từ 5- >6.5 là loại trung bình
 - + < 5 : Loại yếu

Cài đặt chương trình cho phép người dùng nhập thông tin của 1 sinh viên (kiểm tra các ràng buộc phải nhập)

- Xuất ra thông tin của sinh viên vừa nhập (điểm trung bình, xét loại)

Bài 5. Quản lý khóa học

Xây dựng hai lớp Course và CourseManager.

Lớp Course

Thuộc tính có thể gồm:

- Mã khóa học.
- Tên khóa học.
- Số lượng học viên tối đa.
- Danh sách học viên.

Các phương thức:

- `__init__()`: khởi tạo thông tin khóa học.
- `register_student()`: đăng ký học viên nếu còn chỗ.
- `cancel_registration()`: hủy đăng ký.
- `is_full()`: kiểm tra khóa học đã đủ số lượng hay chưa.

Lớp CourseManager

- `add_course()`: thêm khóa học.
- `find_course()`: tìm khóa học theo mã.
- `register_student()`: đăng ký học viên vào khóa học tương ứng.
- `cancel_registration()`: hủy đăng ký.
- `statistics()`: thống kê số học viên của từng khóa học.

Bài 6. Quản lý phòng khách sạn

Xây dựng lớp HotelManager để quản lý các phòng và hoạt động đặt phòng.

Thông tin phòng có thể gồm:

- Số phòng.
- Loại phòng.
- Đơn giá.
- Trạng thái phòng.

Các phương thức:

- `add_room()`: thêm phòng mới.
- `find_available_rooms()`: tìm các phòng đang trống.
- `book_room()`: đặt phòng nếu phòng còn trống.
- `checkout()`: trả phòng và chuyển trạng thái về trống.
- `calculate_payment()`: tính tiền theo đơn giá và số đêm.
- `revenue()`: tính tổng doanh thu từ các lượt lưu trú đã hoàn thành.

Bài 7. Quản lý kho hàng

Xây dựng hai lớp InventoryItem và InventoryManager.

Lớp InventoryItem

Thuộc tính có thể gồm:

- Mã hàng.
- Tên hàng.
- Đơn giá.
- Số lượng tồn.

Các phương thức:

- `import_stock(quantity)`: tăng số lượng tồn khi nhập kho.
- `export_stock(quantity)`: giảm số lượng tồn, không cho xuất quá số lượng hiện có.
- `inventory_value()`: tính giá trị tồn kho của mặt hàng.

Lớp InventoryManager

- `add_item()`: thêm mặt hàng mới.
- `find_item()`: tìm mặt hàng theo mã.
- `import_item()`: nhập hàng.
- `export_item()`: xuất hàng.
- `low_stock_items()`: tìm mặt hàng có số lượng tồn thấp.
- `total_inventory_value()`: tính tổng giá trị hàng hóa trong kho.

Bài 8. Quản lý điểm học phần

Xây dựng hai lớp SubjectResult và GradeManager.

Lớp SubjectResult

Thuộc tính có thể gồm:

- Mã môn học.
- Tên môn học.
- Điểm chuyên cần.
- Điểm giữa kỳ.
- Điểm cuối kỳ.

Các phương thức:

- `__init__()`: khởi tạo thông tin môn học và các điểm.
- `final_score()`: tính điểm tổng kết theo công thức quy định.
- `is_passed()`: xác định môn học đạt hay chưa đạt.
- `display()`: hiển thị kết quả học phần.

Lớp GradeManager

- `add_result()`: thêm kết quả môn học.
- `display_results()`: hiển thị toàn bộ kết quả.
- `average_score()`: tính điểm trung bình các môn.
- `failed_subjects()`: tìm các môn chưa đạt.
- `sort_by_score()`: sắp xếp các môn theo điểm tổng kết.

Bài 9. Hệ thống quản lý bán hàng

Xây dựng các lớp Product, Customer, OrderItem, Order và StoreManager.

Yêu cầu chính:

- Quản lý sản phẩm: thêm, sửa, xóa, tìm kiếm.
- Quản lý khách hàng: thêm và tìm kiếm.
- Tạo đơn hàng và thêm sản phẩm vào đơn.
- Kiểm tra số lượng tồn trước khi bán.
- Tính tổng tiền đơn hàng.
- In hóa đơn.
- Tính doanh thu.
- Thống kê hàng tồn kho.

Bài 10. Hệ thống quản lý lớp học

Xây dựng các lớp Student, Subject và ClassManager.

Yêu cầu chính:

- Thêm sinh viên và môn học.
- Đăng ký môn học cho sinh viên.
- Nhập hoặc cập nhật điểm.
- Tìm sinh viên.
- Tính điểm trung bình.
- Tìm sinh viên đủ điều kiện học bổng.
- Sắp xếp sinh viên theo điểm trung bình.
- Thống kê kết quả học tập của lớp.

Bài 11. Hệ thống quản lý bệnh nhân và lịch khám

Xây dựng các lớp Patient, Doctor, Appointment và ClinicManager.

Yêu cầu chính:

- Thêm bệnh nhân và bác sĩ.
- Tạo lịch khám.
- Kiểm tra bác sĩ không bị trùng lịch cùng thời điểm.
- Hủy lịch khám.
- Tìm lịch theo ngày, bệnh nhân hoặc bác sĩ.
- Cập nhật trạng thái lịch khám.
- Thống kê số lượt khám.
- Tính doanh thu từ các lịch khám đã hoàn thành.